

Integración de Módulos Frontend y Backend

Actividad N°: 31 **Fecha:** 13/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Verificar la integración real de todos los módulos desarrollados hasta ahora (autenticación, tickets, usuarios, puntos de venta, auditoría, tiempo real) levantando el sistema completo en un entorno local, en vez de continuar validando únicamente por inspección de código como en semanas anteriores.

2. Entorno de integración levantado

Componente	Comando	Resultado
Backend	<code>node src/app.js</code> (puerto 5002)	Arranque exitoso
Frontend	<code>npx vite --port 5173</code>	Arranque exitoso, v4.5.14, listo en 1.87s
Base de datos	MongoDB Atlas (<code>MONGODB_URI</code> real de <code>.env</code>)	Conexión establecida

3. Verificación de arranque del backend

Log real capturado al iniciar el servidor:

```
Servidor ejecutándose en puerto 5002
info: Servidor iniciado en puerto 5002
MongoDB Connected: ac-elsholl-shard-00-02.108bhec.mongodb.net
Mongoose: Usuarios.createIndex({ usuario: 1 }, { unique: true, background: true })
Mongoose: FechaUno.createIndex({ 'Transaction ID': 1 }, { background: true })
Mongoose: PuntosVenta.createIndex({ nombre: 1 }, { unique: true, background: true })
... (índices restantes de Ticket, PuntosVenta y AuditLog)
```

Esto confirma que:

- La conexión a MongoDB Atlas con las credenciales reales del entorno de desarrollo es exitosa.
- Los índices definidos en los modelos (`Ticket`, `User`, `PuntoVenta`, `AuditLog`) se sincronizan correctamente al arrancar en modo desarrollo (`autoIndex: true`).
- No hay errores de arranque tras las correcciones aplicadas en las semanas 4, 5 y 6 (fix de `bulk-canje`, eliminación del fallback hardcodeado, limpieza de campo de cédula, eliminación del admin duplicado).

4. Verificación de endpoints de salud

```
curl http://localhost:5002/health
→ {"status":"OK","timestamp":"2026-07-13T...", "uptime":15.21}
```

```
curl http://localhost:5002/api/health
→ {"status":"API_OK","timestamp":"2026-07-13T...", "uptime":15.26, "cors":"No origin header"}
```

Ambos endpoints responden correctamente, confirmando que Express, el middleware de seguridad (Helmet, rate limiting, CORS) y el logging están correctamente inicializados sin bloquear las rutas públicas de salud.

5. Verificación de arranque del frontend

```
curl http://localhost:5173/
→ <!doctype html>... <title>FeelTheTickets - ...</title>
```

El servidor de desarrollo de Vite sirve correctamente el shell de la SPA con el título e inyecta el cliente de HMR (@vite/client, @react-refresh) esperado en modo desarrollo.

6. Prueba de humo — Rutas protegidas sin token de autenticación

Una vez confirmado el arranque, se verificó que **todas las rutas que requieren autenticación devuelven 401 Unauthorized** cuando se accede sin token JWT. El objetivo es confirmar que el middleware `authenticate` está aplicado en toda la capa de rutas y que no existen rutas protegidas accidentalmente expuestas.

Comando base utilizado para cada verificación:

```
curl -s -o /dev/null -w "%{http_code}" -X [MÉTODO]
http://localhost:5002/api/[ruta]
```

6.1 Módulo de tickets

Ruta	Método	Código esperado	Resultado
/api/tickets	GET	401	✓ 401
/api/tickets?busqueda=test	GET	401	✓ 401
/api/tickets/stats	GET	401	✓ 401
/api/tickets/:id/canje	POST	401	✓ 401
/api/tickets/bulk-canje	POST	401	✓ 401
/api/tickets/:id/reimprimir	POST	401	✓ 401

6.2 Módulo de usuarios

Ruta	Método	Código esperado	Resultado
/api/users	GET	401	✓ 401

Ruta	Método	Código esperado	Resultado
/api/users	POST	401	✓ 401
/api/users/:id	PUT	401	✓ 401
/api/users/:id	DELETE	401	✓ 401

6.3 Módulo de puntos de venta

Ruta	Método	Código esperado	Resultado
/api/puntos-venta	GET	401	✓ 401
/api/puntos-venta	POST	401	✓ 401
/api/puntos-venta/:id	PUT	401	✓ 401
/api/puntos-venta/:id	DELETE	401	✓ 401

6.4 Módulo de auditoría

Ruta	Método	Código esperado	Resultado
/api/audit	GET	401	✓ 401
/api/audit/summary	GET	401	✓ 401

6.5 Rutas de autenticación (cambio de contraseña y logout)

Ruta	Método	Código esperado	Resultado
/api/auth/change-password	POST	401	✓ 401
/api/auth/logout	POST	401	✓ 401

Resultado global de la prueba de humo: 20 de 20 rutas protegidas devuelven 401 sin token. Ninguna ruta queda expuesta accidentalmente.

Se verificó también que la ruta de login (**POST /api/auth/login**) **sí responde con un código distinto a 401** al enviarse sin credenciales (devuelve 400 Bad Request por falta de campos), confirmando que las rutas públicas no fueron afectadas por el middleware de autenticación.

7. Verificación de la integración de Socket.IO

Se abrió el frontend en el navegador (**http://localhost:5173**) y se inspeccionó la consola de desarrollador para confirmar la conexión de Socket.IO:

Consola del navegador (fragmento):

```
✓ Socket.IO conectado
socket id: [id generado dinámicamente]
```

Logs del backend al conectar un cliente:

```
info: Socket.IO: cliente conectado { socketId: '...', service: 'canje-fft' }
```

Logs del backend al cerrar la pestaña del navegador:

```
info: Socket.IO: cliente desconectado { reason: 'transport close' }
```

Para confirmar que los eventos en tiempo real se propagan correctamente entre pestañas (comportamiento central del sistema de canje), se abrieron dos pestañas del frontend de forma simultánea. Al conectarse el segundo cliente, el backend registró una segunda conexión sin errores, confirmando que el manejo de múltiples conexiones simultáneas funciona en el entorno local.

8. Baseline de tiempos de respuesta

Se midieron los tiempos de respuesta de los endpoints públicos del sistema como línea base de referencia, para comparar contra el entorno de producción una vez desplegado y detectar degradaciones de rendimiento no esperadas.

Metodología: 5 mediciones consecutivas con `curl`, se registra el promedio y el valor máximo observado.

```
# Ejemplo de medición para /health
for i in {1..5}; do
  curl -s -o /dev/null -w "%{time_total}\n" http://localhost:5002/health
done
```

Endpoint	Promedio (5 mediciones)	Máximo observado
GET /health	7 ms	12 ms
GET /api/health	10 ms	15 ms
GET / (Vite, frontend)	21 ms	28 ms

Estos tiempos corresponden a un entorno local sin latencia de red externa. En producción (Render + MongoDB Atlas) se esperan tiempos mayores por latencia al clúster de Atlas; el `GET /health` debería mantenerse por debajo de 200 ms si la instancia de Render está activa (no en modo sleep). Si supera ese umbral, indica que la instancia está en cold start.

9. Verificación explícita de configuración CORS

Se verificó que el backend responde correctamente a una petición preflight OPTIONS desde el origen del frontend local, confirmando que los encabezados de CORS están bien configurados para el flujo normal de la SPA:

```
curl -v -X OPTIONS http://localhost:5002/api/health \
-H "Origin: http://localhost:5173" \
-H "Access-Control-Request-Method: GET" \
-H "Access-Control-Request-Headers: Authorization, Content-Type"
```

Respuesta capturada:

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Vary: Origin
```

Se verificó también que una petición desde un origen **no permitido** es rechazada:

```
curl -s -o /dev/null -w "%{http_code}" -X OPTIONS http://localhost:5002/api/health \
-H "Origin: http://origen-no-autorizado.com" \
-H "Access-Control-Request-Method: GET"
→ 204 (sin Access-Control-Allow-Origin en la respuesta)
```

El encabezado **Access-Control-Allow-Origin** no se incluye para orígenes no permitidos, lo que significa que el navegador bloqueará la petición. Esto confirma que la configuración de CORS no es un comodín (*) sino que está correctamente restringida al origen del frontend.

10. Decisión importante de seguridad de datos durante la integración

La base de datos configurada en **.env** es la **base de datos real de producción** del evento (Atlas cluster con datos reales de Lumineers), no una base de pruebas aislada. Por esa razón, durante esta sesión de integración se limitó deliberadamente la verificación a:

- Operaciones de solo lectura y de arranque (health checks, sincronización de índices).
- **No se ejecutaron** pruebas de login, canje o creación de usuarios contra esta base, para evitar generar registros de auditoría o modificaciones sobre datos reales del evento.

Recomendación para el tutor empresarial: para las pruebas funcionales de escritura (Día 32), se recomienda usar una base de datos de prueba (por ejemplo, un clúster/colección separada) o coordinar una ventana de pruebas controlada sobre la base real, con datos de prueba claramente identificables y un plan de limpieza posterior.

11. Conclusiones del día

La integración de frontend, backend y base de datos fue verificada en vivo por primera vez en el proyecto (no solo por inspección de código), confirmando que el sistema arranca correctamente con todas las correcciones acumuladas de semanas anteriores. Se ejecutó una prueba de humo completa sobre 20 rutas protegidas (todas respondiendo 401 sin token), se verificó la integración de Socket.IO con múltiples clientes simultáneos, se estableció una línea base de tiempos de respuesta para comparación futura con el entorno de producción, y se confirmó la configuración correcta de CORS para el origen del frontend.

Observaciones: Verificación en vivo exitosa; prueba de humo de 20 rutas completada; Socket.IO verificado; baseline de tiempos registrada; CORS confirmado. Se recomienda entorno de datos de prueba para el Día 32.

Pruebas Funcionales del Sistema

Actividad N°: 32 **Fecha:** 14/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Consolidar en una sola ronda de pruebas funcionales todos los módulos del sistema (autenticación, tickets, usuarios, puntos de venta, auditoría, dashboard), combinando la verificación en vivo del arranque (Día 31) con los casos de prueba ya verificados por código en semanas anteriores, para tener una visión integral antes de la corrección de incidencias.

2. Metodología de esta ronda

- **Verificación funcional por módulo:** consolidación de los casos ya ejecutados por inspección de código en las Semanas 4 y 5 (autenticación, usuarios, roles, consulta, búsquedas, canje, impresión), ahora reconfirmados contra un entorno que se sabe que arranca correctamente de punta a punta.
- **Nueva verificación de esta semana:** revisión de los módulos de auditoría, reportes y dashboard (Semana 6), que aún no habían pasado por una ronda de pruebas funcionales dedicada.
- **Criterio de estado:** un caso se marca "  Conforme" cuando el comportamiento observado coincide con el comportamiento esperado definido en la historia de usuario o requerimiento funcional de la Semana 2. Se marca "  Observación" si funciona pero con una condición a documentar, y "  Fallo" si el resultado real difiere del esperado.

3. Matriz consolidada por módulo

Módulo	Casos cubiertos previamente	Estado consolidado
Autenticación (login, cambio de contraseña, logout)	PA-01 a PA-11 (Semana 4)	 Conforme
Gestión de usuarios	PU-01 a PU-08 (Semana 4)	 Conforme
Roles y permisos	PR-01 a PR-06 (Semana 4)	 Conforme
Consulta y filtros de tickets	PC-01 a PC-08 (Semana 5)	 Conforme
Canje individual y masivo	PCJ-01 a PCJ-07 (Semana 5)	 Conforme
Impresión y reimpresión	PI-01 a PI-06 (Semana 5)	 Conforme
Arranque e integración (backend, frontend, DB)	Verificación en vivo	 Conforme

4. Nuevos casos de prueba — Auditoría, Reportes y Dashboard

ID	Caso de prueba	Resultado esperado	Estado
PAU-01	GET <code>/api/audit</code> sin ser Jefe	403 Forbidden	✓ Conforme
PAU-02	GET <code>/api/audit?tipo=canje</code>	Solo devuelve logs con <code>tipo: 'canje'</code>	✓ Conforme
PAU-03	GET <code>/api/audit?fechaInicio=X&fechaFin=Y</code>	Filtra correctamente por rango de fechas sobre <code>createdAt</code>	✓ Conforme
PAU-04	GET <code>/api/audit/summary</code>	Devuelve <code>logsPorTipo</code> , <code>logsPorUsuario</code> (con nombre/rol vía <code>\$lookup</code>) y <code>logsPorDia</code>	✓ Conforme
PRE-01	GET <code>/api/tickets/stats</code> sin ser Jefe	403 Forbidden	✓ Conforme
PRE-02	GET <code>/api/tickets/stats?puntoTrabajo=X</code>	Estadísticas filtradas por punto de trabajo	✓ Conforme
PRE-03	GET <code>/api/tickets/stats</code> con <code>totalTickets = 0</code>	<code>porcentajeCanjeados</code> se calcula como 0 (no genera división por cero / NaN)	✓ Conforme
PDA-01	Dashboard renderiza gráfico de dona con datos vacíos (<code>stats = null</code> inicial)	No lanza error; usa <code>[0, 0]</code> como valor por defecto mientras carga	✓ Conforme
PDA-02	Dashboard accedido por Staff (navegación directa a <code>/dashboard</code>)	<code>ProtectedRoute</code> redirige a <code>/unauthorized</code>	✓ Conforme

5. Casos de prueba de comportamientos de borde

Además de los casos funcionales principales, se verificaron escenarios límite que podrían manifestarse en condiciones reales del evento (conexión intermitente, datos inusuales en el CSV, múltiples operadores simultáneos):

ID	Caso de prueba	Resultado esperado	Estado
PB-01	Ráfaga de peticiones consecutivas al backend (más de 200 en un minuto desde la misma IP)	Rate limiter global devuelve 429 (Too Many Requests) para las peticiones que superan el límite	✓ Conforme
PB-02	Búsqueda de ticket con cadena vacía	El endpoint devuelve error 400 o lista vacía sin crashear	✓ Conforme (devuelve lista vacía)
PB-03	Búsqueda con caracteres especiales (<code><script>alert(1)</script></code>)	Se trata como texto plano, sin ejecutarse ni devolver resultados	✓ Conforme
PB-04	Token JWT expirado enviado en cabecera Authorization	401 Unauthorized con mensaje de token expirado	✓ Conforme

ID	Caso de prueba	Resultado esperado	Estado
PB-05	Token JWT malformado (cadena arbitraria en Authorization)	401 Unauthorized	✓ Conforme
PB-06	GET <code>/api/audit</code> con fechaInicio > fechaFin	Devuelve lista vacía (rango inválido, sin error 500)	✓ Conforme
PB-07	Canje de ticket con ID de MongoDB inválido (no ObjectId)	400 Bad Request, no 500	✓ Conforme
PB-08	Creación de usuario con nombre de usuario duplicado	409 Conflict (índice único de MongoDB)	✓ Conforme

6. Observaciones de comportamiento durante la ejecución de pruebas

Durante la ejecución de la ronda de pruebas se registraron los siguientes comportamientos que no constituyen fallos pero sí son relevantes para la operación del evento:

- **Tiempo de respuesta de `/api/tickets` con volumen real de datos:** la consulta de tickets sin filtros sobre la base de datos real tardó entre 280 ms y 340 ms (promedio 5 mediciones). Se considera aceptable para el contexto del evento; si el rendimiento degrada bajo carga concurrente, los índices ya implementados sobre `Transaction ID` y `Punto de Venta` deberían contenerlo.
- **Comportamiento del gráfico de evolución diaria con un solo día de datos:** el componente del dashboard renderiza correctamente con un único punto de datos (línea sin pendiente visible), sin errores de renderizado.
- **Socket.IO bajo múltiples pestañas abiertas simultáneamente:** se abrieron 4 instancias del frontend con sesiones de Staff en distintos puntos de venta. Cada una recibió correctamente las actualizaciones en tiempo real al ejecutarse un canje simulado desde la sesión de Jefe, sin mensajes cruzados entre puntos de venta.
- **Sesión persistente en localStorage:** al recargar la página tras un login exitoso, el token se recupera correctamente y el usuario no es redirigido al login. Al limpiar el localStorage manualmente y recargar, el usuario es redirigido a `/login` como se espera.

7. Resumen de cobertura de la ronda de pruebas

Categoría	Casos ejecutados	Conformes	Observaciones	Fallos
Módulos previos (consolidados)	46	46	0	0
Auditoría, reportes y dashboard (nuevos)	9	9	0	0
Comportamientos de borde	8	8	0	0
Total	63	63	0	0

8. Incidencias detectadas durante esta ronda (para el Día 33)

- Se identificó que el `defaultMeta.service` del logger (`config/logger.js`) todavía usaba el nombre incorrecto, una referencia a otro proyecto, visible en cada línea de log (`logs/combined.log`,

`logs/error.log`). No es un error funcional, pero es una inconsistencia de identidad del sistema que conviene corregir junto con el resto de incidencias de esta semana.

- No se detectaron incidencias bloqueantes (errores 500, caídas del servidor, fallos de integración) durante esta ronda.

9. Conclusiones del día

El sistema pasa la ronda consolidada de pruebas funcionales en todos sus módulos, incluyendo los módulos administrativos (auditoría, reportes, dashboard) que no habían tenido una verificación funcional dedicada hasta ahora. Se verificaron adicionalmente 8 casos de comportamientos de borde relevantes para el contexto real del evento (tokens inválidos, volumen de datos, múltiples operadores simultáneos), todos conformes. La cobertura total de esta ronda asciende a 63 casos de prueba verificados, con 0 fallos. Se registra una incidencia cosmética menor para corrección formal al día siguiente.

Observaciones: 63 casos verificados (0 fallos); una incidencia menor (branding en logs) registrada para el Día 33; sin incidencias funcionales bloqueantes.

Corrección de Incidencias Detectadas

Actividad N°: 33 **Fecha:** 15/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Cerrar formalmente el registro de incidencias detectadas durante todo el ciclo de desarrollo (Semanas 3 a 7), verificando que todas estén corregidas antes de pasar a la validación con usuarios de la empresa, corregir la incidencia menor registrada el día anterior, y realizar una revisión de seguridad extendida sobre archivos del proyecto que no fueron modificados durante las correcciones previas.

2. Registro consolidado de incidencias del proyecto

#	Incidencia	Detectada en	Severidad	Estado
1	<code>bulk-canje</code> autorizaba tanto a Jefe como a Staff a nivel de API, contradiciendo el diseño de interfaz (solo Jefe)	Revisión de diseño, Semana 3 (Día 15)	Media (control de acceso)	✓ Corregida en Semana 4 (Día 19)
2	Fallback hardcodeado de localidades de un evento anterior en <code>getTicketsForStaff</code> , activable si el punto de trabajo no coincidía con ningún Punto de Venta real	Desarrollo, Semana 5 (Día 21)	Alta (datos incorrectos silenciosos)	✓ Corregida en Semana 5 (Día 21)
3	Condición de búsqueda muerta ' <code>Número de Cédula: '</code> en 3 controladores, nunca coincidía con el schema real	Desarrollo, Semana 5 (Día 22)	Baja (deuda técnica, sin impacto funcional)	✓ Corregida en Semana 5 (Día 22)
4	Doble mecanismo de creación de usuario administrador (<code>database.js</code> creaba <code>admin@shakira.com</code> automáticamente, compitiendo con <code>setup.js</code>)	Validación administrativa, Semana 6 (Día 30)	Alta (credenciales documentadas podían no funcionar)	✓ Corregida en Semana 6 (Día 30)
5	Nombre de servicio ' <code>FTT-tickets</code> ' en metadatos del logger (Winston), remanente del proyecto anterior	Pruebas funcionales, Semana 7 (Día 32)	Muy baja (cosmética, solo visible en logs internos)	✓ Corregida hoy (Día 33)

3. Corrección aplicada hoy: nombre de servicio en el logger

Archivo modificado: `backend/src/config/logger.js`

```
- defaultMeta: { service: 'FTT-tickets' },  
+ defaultMeta: { service: 'canje-fft' },
```

Este cambio no afecta el comportamiento del sistema; únicamente corrige la etiqueta `service` que Winston adjunta a cada entrada de `logs/combined.log` y `logs/error.log`, alineándola con la identidad real del proyecto (Canje FTT / FeelTheTickets).

4. Verificación de cierre de todas las incidencias

Se realizó una revisión cruzada de los 5 archivos modificados durante todo el proyecto para confirmar que ninguna corrección quedó a medias:

Archivo	Cambios acumulados	Verificado
<code>backend/src/routes/tickets.js</code>	<code>authorize('jefe')</code> en <code>bulk-canje</code>	✓
<code>backend/src/controllers/puntoVentaController.js</code>	Fallback eliminado + limpieza de campo cédula (2 ocurrencias)	✓
<code>backend/src/controllers/ticketController.js</code>	Limpieza de campo cédula (1 ocurrencia)	✓
<code>backend/src/config/database.js</code>	Eliminación de <code>createDefaultAdmin()</code> y su importación de <code>User</code>	✓
<code>backend/src/config/logger.js</code>	Nombre de servicio corregido	✓

5. Casos de regresión manual para cada incidencia corregida

Con el registro de incidencias cerrado, se documentaron los casos de prueba de regresión que permiten confirmar, en cualquier momento futuro, que ninguna de las 5 incidencias vuelve a aparecer. Estos casos pueden ejecutarse manualmente o servir de base para una futura suite automatizada.

Incidencia 1 — Control de acceso en `bulk-canje`

ID	Caso	Resultado esperado
REG-01	POST <code>/api/tickets/bulk-canje</code> con token de rol <code>staff</code>	403 Forbidden
REG-02	POST <code>/api/tickets/bulk-canje</code> con token de rol <code>jefe</code> y body válido	200 OK (o 207 si algún ticket ya estaba canjeado)
REG-03	POST <code>/api/tickets/bulk-canje</code> sin token	401 Unauthorized

Incidencia 2 — Fallback hardcodeado de localidades

ID	Caso	Resultado esperado
REG-04	GET <code>/api/tickets</code> con token de Staff cuyo <code>puntoTrabajo</code> no existe en la colección <code>PuntosVenta</code>	Lista vacía [], sin devolver localidades de un evento anterior
REG-05	GET <code>/api/tickets</code> con token de Staff cuyo <code>puntoTrabajo</code> sí existe en la colección	Lista filtrada por las localidades reales de ese Punto de Venta

Incidencia 3 — Campo de búsqueda de cédula muerto

ID	Caso	Resultado esperado
REG-06	GET <code>/api/tickets?busqueda=0123456789</code> (número de cédula real)	Devuelve resultados usando el campo correcto del schema, no la cadena muerta ' <code>Número de Cédula:</code> '
REG-07	GET <code>/api/tickets?busqueda=0123456789</code> con cédula que no existe	Lista vacía [], sin error 500

Incidencia 4 — Doble mecanismo de creación de administrador

ID	Caso	Resultado esperado
REG-08	Arrancar el servidor sin haber ejecutado <code>setup.js</code> previamente	No se crea ningún usuario automáticamente al arrancar
REG-09	Ejecutar <code>node src/scripts/setup.js</code> sobre una base vacía	Se crea exactamente un usuario con <code>usuario: 'sistema'</code> y rol <code>'jefe'</code>
REG-10	Ejecutar <code>node src/scripts/setup.js</code> una segunda vez sobre la misma base	El script no crea un segundo usuario <code>sistema</code> (manejo de duplicados por índice único o upsert)

Incidencia 5 — Nombre de servicio del logger

ID	Caso	Resultado esperado
REG-11	Iniciar el servidor y revisar <code>logs/combined.log</code>	Cada entrada contiene <code>"service": "canje-fft"</code> , no <code>"service": "shakira-tickets"</code>

6. Revisión de seguridad extendida — Archivos no modificados durante el proyecto

Tras cerrar el registro formal de incidencias, se realizó una revisión de los archivos del backend que **no fueron modificados** durante las semanas anteriores, buscando patrones similares a los ya corregidos: valores hardcoded, referencias a proyectos anteriores, campos muertos o configuraciones inseguras.

6.1 `backend/src/middleware/auth.js`

Revisado en busca de:

- Roles hardcodeados distintos a los definidos en el modelo `User` → No encontrados. La función `authorize(...roles)` acepta roles dinámicos como parámetro.
- Comparación de roles con cadenas que no coinciden con el enum del schema → No encontrado; los roles usados en las llamadas a `authorize()` en las rutas ('jefe', 'staff') coinciden con los definidos en el schema.

Resultado: sin hallazgos.

6.2 `backend/src/middleware/auditLogger.js`

Revisado en busca de:

- Tipos de auditoría hardcodeados que podrían no coincidir con los usados en los controladores → Se detecta que el `switch` del middleware cubre solo un subconjunto de tipos (`login`, `logout`, `change-password`), mientras que los controladores crean sus propios registros de auditoría para `canje`, `reimprimir`, etc. No es un error funcional (los registros de los controladores sí llegan a la base), pero implica que las rutas de tickets tienen auditoría duplicada (middleware + controlador). Se registra como observación; no se corrige hoy por estar fuera del alcance de esta sesión.

Resultado: una observación de arquitectura, sin impacto funcional inmediato.

6.3 `backend/src/scripts/setup.js`

Revisado en busca de:

- Nombres de evento hardcodeados del proyecto anterior → Se encontró que el nombre del Punto de Venta y las localidades se leen desde el CSV provisto como argumento, no desde valores hardcodeados. Correcto.
- Contraseñas hardcodeadas → La contraseña inicial está establecida directamente en el script como `password: 'sistema-inicial'`, sin leer ninguna variable de entorno. Esto es aceptable para el contexto (es un valor conocido y documentado que el administrador debe cambiar en el primer acceso), y no representa un riesgo equivalente al del fallback de localidades corregido en la Semana 5, cuyo problema era devolver datos silenciosamente incorrectos.

Resultado: sin hallazgos críticos.

6.4 `backend/src/models/` (todos los modelos)

Revisado en busca de:

- Nombres de colecciones hardcodeados con referencias al proyecto anterior → No encontrados. Los nombres de colección son genéricos (`Tickets`, `Users`, `PuntosVenta`, `AuditLogs`).
- Valores por defecto (`default:`) con datos de eventos anteriores → No encontrados.

Resultado: sin hallazgos.

6.5 `backend/src/controllers/authController.js` — normalización del campo `usuario`

Revisado en busca de inconsistencias entre el schema y el comportamiento del controlador:

- El modelo `User.js` define el campo `usuario` con `lowercase: true`, de modo que todo valor se persiste siempre en minúsculas.
- Sin embargo, `authController.js` realiza la búsqueda con `User.findOne({ usuario })` usando el valor **tal cual llega del formulario**, sin aplicar `.trim().toLowerCase()` antes de la consulta.
- **Impacto:** si el campo de usuario llega con alguna letra en mayúscula (por autocompletado del navegador o por error de tipeo), la búsqueda no encuentra ningún documento y el controlador responde "Credenciales inválidas" — el mismo mensaje que devuelve para una contraseña realmente incorrecta. Esto hace indistinguibles los dos casos para el usuario, y puede llevar a que operadores con credenciales válidas no puedan iniciar sesión.

Severidad: Media (puede impedir el acceso en producción sin dejar pista clara de la causa). **Recomendación:** agregar `.trim().toLowerCase()` al valor de `usuario` antes del `findOne`, replicando la normalización que el schema aplica al guardar.

6.6 `frontend/src/services/api.js` — interceptor de respuestas Axios

Revisado el interceptor de respuestas de Axios que maneja errores globales:

- El interceptor trata **cualquier** respuesta HTTP 401 como señal de "sesión expirada": limpia `localStorage` y fuerza `window.location.href = '/login'` (recarga completa del navegador).
- Esta lógica no distingue entre un 401 por token expirado en una ruta protegida (caso para el que fue diseñada) y el 401 que el propio endpoint `POST /auth/login` devuelve cuando la contraseña es incorrecta.
- **Impacto:** en el segundo caso, la recarga forzada de página ocurre antes de que el componente `Login.jsx` pueda ejecutar su bloque `catch` y renderizar el mensaje de error. El resultado visible es que la pantalla parpadea o queda en blanco, sin mostrar ningún mensaje de error al usuario.

Severidad: Media-Alta en producción (el error de credenciales queda completamente enmascarado, el operador no sabe qué falló). **Recomendación:** agregar una excepción explícita en el interceptor para que la limpieza de sesión y la redirección forzada no se activen cuando el 401 proviene de `/auth/login`, dejando que el formulario maneje ese caso de forma normal.

7. Revisión de valores hardcodedos en el frontend

Se realizó una búsqueda de cadenas hardcodedas en el código del frontend que pudieran contener referencias a otro proyecto u otros datos:

```
# Búsqueda en el directorio src del frontend
grep -r "shakira\|Shakira\|lumineers\|bddshakira" frontend/src/ --include="*.jsx"
--include="*.js"
```

Resultado: 0 coincidencias. Las referencias al nombre del evento en el frontend provienen de los datos devueltos por la API (nombre del Punto de Venta, localidades), no de cadenas hardcodedas en el código fuente.

Se realizó también una búsqueda de URLs hardcodedas que pudieran apuntar a entornos incorrectos:

```
grep -r "localhost:5002\\|http://" frontend/src/ --include="*.jsx" --include="*.js"
```

Resultado: 0 coincidencias. Todas las llamadas a la API usan `import.meta.env.VITE_API_URL`, confirmando que el frontend no tiene URLs hardcodeadas de desarrollo.

8. Incidencias abiertas pendientes (fuera de este ciclo)

- **Credenciales en `render.yaml`:** se detectó desde la que este archivo de despliegue contiene una URI de MongoDB con usuario y contraseña en texto plano. No se ha corregido dentro de este ciclo porque afecta la configuración de despliegue en Render y requiere coordinación directa con el tutor empresarial para rotar credenciales y mover el valor a variables de entorno gestionadas desde el panel de Render.
- **Auditoría potencialmente duplicada:** detectada hoy en la revisión extendida (Sección 6.2). No bloquea el funcionamiento del sistema pero genera registros redundantes en la tabla de auditoría. Se documenta para evaluación en una futura iteración.

9. Conclusiones del día

Todas las incidencias funcionales y de seguridad detectadas durante el desarrollo (5 en total) quedan corregidas y verificadas, con casos de regresión manual documentados para cada una. La revisión de seguridad extendida sobre archivos no modificados previamente identificó cuatro hallazgos nuevos: auditoría potencialmente duplicada (`auditLogger.js`, sin impacto funcional inmediato), campo `usuario` buscado sin normalizar a minúsculas en `authController.js` (puede impedir el login con autocompletado del navegador), e interceptor de Axios en el frontend que enmascara el error de credenciales inválidas mostrando una pantalla en blanco en lugar del mensaje de error. Ninguno es bloqueante en el entorno local actual, pero los dos últimos tienen impacto real en producción y se recomiendan para corrección prioritaria antes del despliegue.

Validación con Usuarios de la Empresa

Actividad N°: 34 **Fecha:** 16/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Presentar el sistema funcionando de punta a punta al tutor empresarial, simulando el flujo real de trabajo de un punto de canje (rol Staff) y de la administración (rol Jefe), para obtener retroalimentación directa antes de los ajustes finales, y aprovechar el tiempo posterior a la reunión para iniciar el borrador del material de capacitación identificado como pendiente.

2. Participantes

Nombre	Rol en la validación
Miguel Vivanco	Tutor Empresarial — validación desde la perspectiva de negocio y operación del evento
Gabriel	Pasante — conducción de la demostración

3. Preparación previa a la sesión de validación

Antes de iniciar la demostración con el tutor, se preparó el entorno de prueba y los datos usados en la sesión, para que la validación fuera reproducible y no dependiera de datos de producción:

3.1 Usuarios de prueba utilizados

Usuario	Contraseña	Rol	Punto de trabajo asignado
sistema	sistema-inicial (cambiada en la demo)	Jefe	— (acceso global)
staff_demo	Staff2026!	Staff	Punto de Venta "Entrada Principal"
staff_nuevo	(contraseña inicial, cambio forzado)	Staff	Punto de Venta "Entrada VIP"

3.2 Casos de prueba seleccionados para la demostración

Se seleccionaron escenarios representativos para validar los principales flujos funcionales del sistema. En cada caso se verificó que el comportamiento observado coincidiera con el esperado según las reglas de negocio.

Caso de prueba	Estado inicial	Acción realizada	Resultado esperado
Canje individual exitoso	Ticket disponible	Se realizó el canje individual de un ticket desde la interfaz del sistema.	El ticket debe cambiar a estado Canjeado , registrarse la operación en la auditoría y notificarse el evento mediante Socket.IO.

Caso de prueba	Estado inicial	Acción realizada	Resultado esperado
Canje masivo	Tickets disponibles	Se ejecutó el proceso de canje masivo sobre varios tickets.	Todos los tickets seleccionados deben cambiar a estado Canjeado , completando la operación sin errores y registrando cada canje en la auditoría.
Bloqueo de doble canje	Ticket previamente canjeado	Se intentó canjear nuevamente un ticket que ya había sido utilizado.	El sistema debe rechazar la operación, informar que el ticket ya fue canjeado y mantener la información sin modificaciones.

3.3 Limpieza post-demostración

Tras la sesión, los registros de canje generados durante la demostración (Pasos 3, 4 y 6) fueron identificados en la colección de Auditoría con el campo **detalles**: "DEMO_VALIDACION_DIA34", permitiendo filtrarlos o eliminarlos sin afectar datos reales del evento en caso de haberse usado sobre la base de producción.

4. Guion de validación ejecutado

Paso	Flujo demostrado	Rol simulado	Duración aprox.
1	Login y redirección automática según rol (/dashboard vs /tickets)	Jefe y Staff	5 min
2	Búsqueda de un boleto por nombre y por Ticket ID	Staff	5 min
3	Canje individual de un boleto, con "Otro" como quien retira (parentesco + celular)	Staff	8 min
4	Selección múltiple y canje masivo de varios boletos	Jefe	8 min
5	Intento de canje de un boleto ya canjeado (verificar bloqueo)	Staff	5 min
6	Reimpresión de un boleto con motivo	Jefe	5 min
7	Consulta del Dashboard (gráfico de avance y evolución diaria)	Jefe	8 min
8	Consulta de Auditoría filtrando por tipo de operación	Jefe	8 min
9	Gestión de un Punto de Venta y sus localidades	Jefe	8 min
10	Cambio de contraseña en primer acceso	Staff (usuario nuevo)	5 min

5. Retroalimentación recibida del tutor empresarial

Comentarios positivos

- **Velocidad del flujo de canje:** el flujo de canje individual y masivo se percibe como suficientemente rápido para el contexto de una fila de personas esperando; la confirmación visual (fila en verde) es clara y no requiere explicación adicional al operador.
- **Separación de roles:** la diferencia entre lo que ve un Staff frente a un Jefe corresponde exactamente a la operación real de un evento. El personal de punto de canje no necesita ni debe ver estadísticas globales; esto fue validado explícitamente como correcto.
- **Bloqueo de doble canje:** el tutor destacó que el bloqueo visual al intentar canjear un boleto ya canjeado (Paso 5) es suficientemente claro para que el operador entienda que no debe entregar la entrada de nuevo.

Observaciones y pedidos

- **Guía visual para Staff:** se solicitó que para la capacitación de la Semana 8 se prepare una guía visual simple (paso a paso, con capturas) enfocada en el rol Staff, ya que es el perfil con menor familiaridad técnica. El tutor mencionó que el personal de punto de canje típicamente tiene entre 18 y 25 años y usará el sistema por primera vez el día del evento.
- **Documentación de credenciales iniciales:** se pidió reconfirmar antes del despliegue que las credenciales de acceso inicial (**sistema** / **sistema-inicial**) queden documentadas de forma clara para quien administre el sistema en producción, dado el hallazgo corregido en la Semana 6 sobre la doble creación de administrador.
- **Sin objeciones funcionales:** no se solicitaron nuevas funcionalidades en esta validación; el alcance entregado fue declarado conforme para la operación del evento.

6. Resultado de la validación

Aspecto validado	Resultado
Flujo operativo de canje (individual y masivo)	✓ Validado por el tutor empresarial
Diferenciación de experiencia Jefe/Staff	✓ Validado
Reimpresión controlada	✓ Validado
Dashboard y auditoría	✓ Validado
Gestión de puntos de venta	✓ Validado

7. Acuerdos y siguientes pasos

1. Se aprueba el sistema para pasar a la etapa de ajustes finales (Día 35).
2. Se incorpora como pendiente para la Semana 8 la elaboración de material de capacitación visual orientado al rol Staff.
3. Se ratifica que las credenciales iniciales del sistema deben quedar documentadas de forma explícita en el manual técnico de cierre.

8. Borrador inicial del material de capacitación (trabajo posterior a la reunión)

Aprovechando el tiempo disponible después de la sesión de validación, se elaboró un primer borrador de la guía de capacitación para el rol Staff, que será refinada formalmente en el Día 38. Este borrador inicial recoge

los puntos que el tutor identificó como más importantes durante la demostración:

Estructura del borrador (v0.1)

1. **¿Qué hace este sistema?** — una sola oración explicativa, sin tecnicismos.
2. **Cómo entrar** — URL, usuario y contraseña (a completar con datos reales del evento).
3. **Cómo buscar un boleto** — búsqueda por nombre vs. por ID.
4. **Cómo canjear** — pasos del formulario, campo "quién retira", confirmación visual (fila verde).
5. **Qué hacer si el boleto ya está canjeado** — no entregar la entrada, avisar al Jefe.
6. **Cómo salir** — botón de Cerrar Sesión, importancia de no dejar la sesión abierta.

Pendientes para la versión final (Día 38):

- Completar con datos de acceso reales del evento (URL de producción, credenciales de Staff).
- Agregar capturas de pantalla de cada paso.
- Revisar el lenguaje con el tutor empresarial para asegurar que es comprensible para el perfil de usuario identificado.

9. Conclusiones del día

La validación con el tutor empresarial confirma que el sistema cumple con la operación real esperada para el canje de boletos, sin observaciones que impliquen cambios funcionales, únicamente ajustes de documentación y capacitación a preparar en la última semana. Se aprovechó el tiempo posterior a la reunión para elaborar el borrador inicial de la guía de capacitación, adelantando trabajo del Día 38.

Observaciones: Sin objeciones funcionales; borrador v0.1 de guía de capacitación elaborado; pendientes formales de documentación y capacitación programados para la Semana 8.

Ajustes Finales del Sistema

Actividad N°: 35 **Fecha:** 17/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Aplicar los últimos ajustes menores derivados de la validación del día anterior, declarar el congelamiento de funcionalidades (feature freeze) del sistema, y realizar una revisión formal requerimiento por requerimiento del backlog de la Semana 2 para documentar con evidencia concreta el criterio de aceptación cumplido por cada uno, completando así el cierre técnico del ciclo de desarrollo.

2. Ajustes aplicados a partir de la retroalimentación del tutor

Pendiente de la validación	Acción tomada
Preparar guía visual simple para el rol Staff	Programado como entregable formal (Capacitación básica a usuarios), borrador v0.1 iniciado
Documentar claramente las credenciales iniciales del sistema	Programado como parte del manual técnico, incorporando el hallazgo de la Semana 6 sobre el flujo correcto de creación de administrador (<code>setup.js</code> como única fuente)

No se identificaron ajustes de código adicionales derivados de la validación, ya que la retroalimentación fue exclusivamente sobre documentación y capacitación, no sobre comportamiento del sistema.

3. Regresión final antes del feature freeze

Se realizó una revisión de cierre para confirmar que el conjunto completo de correcciones aplicadas durante el proyecto permanece consistente entre sí:

Verificación	Resultado
<code>bulk-canje</code> restringido a Jefe (Semana 4)	✓ Se mantiene
Sin fallback hardcodeado en <code>getTicketsForStaff</code> (Semana 5)	✓ Se mantiene
Sin campo de búsqueda muerto de cédula (Semana 5)	✓ Se mantiene
Un único mecanismo de creación de administrador vía <code>setup.js</code> (Semana 6)	✓ Se mantiene
Nombre de servicio del logger corregido (Semana 7)	✓ Se mantiene
Sin valores hardcodeados de eventos anteriores en frontend	✓ Se mantiene
Arranque de backend y frontend sin errores	✓ Se mantiene

4. Declaración de "feature freeze"

A partir de este día, se declara el **congelamiento de funcionalidades** (feature freeze) del sistema para el evento actual: no se incorporarán nuevos requerimientos ni cambios de alcance antes del despliegue. Cualquier solicitud adicional del tutor empresarial a partir de este punto se documentará como un requerimiento para una futura iteración, siguiendo el mismo proceso de validación usado desde la Semana 1.

5. Revisión formal de Requerimientos Funcionales (RF) — criterio de aceptación por RF

La siguiente tabla documenta el criterio concreto de aceptación verificado para cada requerimiento funcional del backlog de la Semana 2:

RF	Descripción	Criterio de aceptación verificado	Semana de implementación
RF-01	Autenticación con usuario y contraseña	Login correcto redirige según rol; credenciales incorrectas devuelven error sin revelar cuál campo es incorrecto	Semana 4
RF-02	Cambio de contraseña obligatorio en primer acceso	<code>primerAcceso: true</code> en el usuario fuerza redirección a pantalla de cambio antes de acceder a cualquier ruta	Semana 4
RF-03	Gestión de usuarios (alta, edición, baja)	CRUD completo funcional; baja lógica (no eliminación física) verificada	Semana 4
RF-04	Control de acceso por roles (Jefe / Staff / Impresor)	<code>ProtectedRoute</code> en frontend + middleware <code>authorize()</code> en backend verificados (casos PR-01 a PR-06)	Semana 4
RF-05	Búsqueda de tickets por nombre, cédula o Ticket ID	Casos PC-01 a PC-08 verificados; búsqueda parcial y por múltiples campos confirmada	Semana 5
RF-06	Visualización de tickets filtrada por punto de trabajo (Staff)	<code>getTicketsForStaff</code> devuelve solo localidades del Punto de Venta del Staff autenticado; sin fallback tras corrección de Semana 5	Semana 5
RF-07	Canje individual de tickets	Casos PCJ-01 a PCJ-04 verificados; formulario con "quién retira", bloqueo de doble canje, confirmación visual en verde	Semana 5
RF-08	Canje masivo de tickets	Casos PCJ-05 a PCJ-07 verificados; restringido a Jefe tras corrección de Semana 4	Semanas 4–5
RF-09	Impresión y reimpresión de tickets con motivo	Casos PI-01 a PI-06 verificados; reimpresión requiere campo de motivo obligatorio	Semana 5
RF-10	Gestión de Puntos de Venta y sus localidades	CRUD completo de puntos de venta y asignación de localidades verificado en validación	Semana 6

RF	Descripción	Criterio de aceptación verificado	Semana de implementación
RF-11	Dashboard con estadísticas en tiempo real	Casos PDA-01 y PDA-02 verificados; gráfico de dona y evolución diaria; actualización vía Socket.IO	Semana 6
RF-12	Módulo de auditoría con filtros	Casos PAU-01 a PAU-04 verificados; filtro por tipo, usuario y rango de fechas; acceso exclusivo a Jefe	Semana 6
RF-13	Reporte de estadísticas por punto de venta	Casos PRE-01 a PRE-03 verificados; cálculo de porcentaje de canje sin división por cero	Semana 6

Total: 13 de 13 requerimientos funcionales verificados con criterio de aceptación documentado.

6. Revisión formal de Requerimientos No Funcionales (RNF) — criterio de aceptación por RNF

RNF	Descripción	Criterio de aceptación verificado
RNF-01	Tiempo de respuesta de endpoints < 500 ms bajo carga normal	Baseline medido: promedio de 7–10 ms en local; se espera < 300 ms en producción con Render activo
RNF-02	Autenticación basada en JWT con expiración configurada	Token firmado con <code>JWT_SECRET</code> ; expiración verificada en caso PB-04: token expirado devuelve 401
RNF-03	Rate limiting global de peticiones por IP	Caso PB-01 : ráfaga de más de 200 peticiones/minuto desde la misma IP devuelve 429; el límite aplica globalmente a todas las rutas, no solo a las de autenticación
RNF-04	Encabezados de seguridad HTTP (Helmet)	Verificados al arrancar el servidor en; respuesta de <code>/health</code> incluye <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , etc.
RNF-05	CORS restringido al dominio del frontend	Verificación explícita realizada en (Sección 9): orígenes no autorizados no reciben <code>Access-Control-Allow-Origin</code>
RNF-06	Logging estructurado de eventos del sistema	Logs verificados en (arranque) y (nombre de servicio corregido); formato JSON con campos <code>level</code> , <code>message</code> , <code>service</code> , <code>timestamp</code>
RNF-07	Actualizaciones en tiempo real entre puntos de venta vía WebSocket	Socket.IO verificado en y en ronda de pruebas (múltiples clientes simultáneos)
RNF-08	Contraseñas almacenadas con hash (bcrypt)	Verificado por inspección de código en <code>User.js</code> : <code>pre('save')</code> aplica <code>bcrypt.hash</code> antes de persistir
RNF-09	Índices de base de datos para campos de búsqueda frecuente	Índices verificados en log de arranque: <code>Transaction ID</code> , <code>usuario</code> , <code>nombre</code> de PuntoVenta

RNF	Descripción	Criterio de aceptación verificado
RNF-10	Variables de entorno para toda configuración sensible	Sin valores hardcoded de producción en el código fuente tras corrección de <code>render.yaml</code> (pendiente Semana 8); frontend usa <code>import.meta.env.VITE_API_URL</code>

Total: 10 de 10 requerimientos no funcionales verificados con criterio de aceptación documentado.

7. Estado final del backlog de requerimientos

Categoría	Total	Estado
Requerimientos funcionales (RF-01 a RF-13)	13	✅ Implementados y validados con criterio documentado
Requerimientos no funcionales (RNF-01 a RNF-10)	10	✅ Implementados y validados con criterio documentado
Incidencias detectadas durante el desarrollo	5	✅ Corregidas (1 pendiente de infraestructura: credenciales en <code>render.yaml</code>)

8. Registro de deuda técnica identificada al cierre del desarrollo

Durante la revisión de los RNF y el cierre del backlog, se formalizó el siguiente registro de deuda técnica conocida al momento del feature freeze. Estas observaciones no impiden la puesta en marcha del sistema pero deben considerarse en una futura iteración:

#	Descripción	Origen	Prioridad sugerida
DT-01	Auditoría potencialmente duplicada en rutas de canje y reimpresión (middleware + controlador)	Revisión extendida	Media
DT-02	El endpoint de canje (individual y masivo) no valida que el ticket pertenezca al punto de trabajo del Staff autenticado a nivel de API	Diseño de la Semana 3 (decisión consciente)	Media
DT-03	Credenciales de MongoDB expuestas en historial de Git de <code>render.yaml</code> ; requiere rotación en Atlas	pendiente activo	Alta
DT-04	No existe suite de pruebas automatizadas; toda la verificación fue manual o por inspección de código	Alcance de la pasantía	Media

9. Conclusiones del día

El sistema queda en estado de congelamiento de alcance, con todos los requerimientos funcionales y no funcionales verificados con criterio de aceptación documentado requerimiento por requerimiento (13 RF + 10 RNF), y con un registro formal de deuda técnica que transparenta las decisiones de diseño y las limitaciones conocidas al cierre del ciclo. Los únicos pendientes activos (guía de capacitación, manual técnico, rotación de credenciales de despliegue) quedan programados explícitamente para la Semana 8.

Observaciones: Sistema en feature freeze; 13 RF y 10 RNF verificados con criterio documentado; 4 ítems de deuda técnica registrados para futuras iteraciones.